

Documento Técnico de Diseño Conceptual y Lógico — Sistema Ecommify: Arquitectura Híbrida PostgreSQL + MongoDB sobre el Dataset Brazilian E-Commerce de Olist

Equipo Desarrollo

Fredy Orlando Pulido Quintero

Myriam Andrea Martinez Fontecha

Nicolas Felipe Torres Amaya

Juan Francisco Javier Perez Rivero

Roles a ejecutar:

Arquitecto de Datos / DBA /Ingeniero de Datos / DevOps

Repositorio: <https://github.com/insoftcol/optimizacion-y-diseno-base-de-datos-v2/upload/main>

Enfoque Arquitectónico

Persistencia políglota híbrida — PostgreSQL 16 (CP) + MongoDB 7.x (AP)

Maestría en Arquitectura de Software

Universidad de La Sabana · Mayo de 2026

Resumen Ejecutivo

Este documento consolida el diseño conceptual y lógico de Ecommify, una plataforma de comercio electrónico que adopta una arquitectura de persistencia políglota híbrida basada en PostgreSQL 16 y MongoDB 7.x. El núcleo transaccional (orders, order_items, order_payments, customers, sellers, products) reside en PostgreSQL bajo posición CAP de Consistencia y Tolerancia a Particiones (CP), normalizado hasta 3FN/FNBC, con replicación streaming sincrónica y particionamiento RANGE anual de la tabla de hechos orders. Los dominios de esquema flexible (reviews) y geoespacial masivo (geolocation) residen en MongoDB bajo posición de Disponibilidad y Tolerancia a Particiones (AP), con replica sets y consistencia eventual. Una colección analítica orders_summary alimentada por un pipeline ETL batch y una colección products_catalog desnormalizada complementan el modelo. El diseño incorpora tipos avanzados PostgreSQL (JSONB para product_specifications, TEXT[] para photos, TSTZRANGE para promotion_period), extensiones (PostGIS para distancia cliente-depósito, pg_trgm para búsqueda fuzzy, pgcrypto), y técnicas OLTP/OLAP híbridas (vistas materializadas semanales, triggers updated_at, jobs VACUUM diarios). El alcance académico opera sobre Supabase plan gratuito y MongoDB Atlas M0; las limitaciones se mitigan mediante muestreo de geolocation y sharding emulado en Docker local.

Palabras clave: Ecommify, PostgreSQL, MongoDB, persistencia políglota, normalización 3FN, JSONB, particionamiento, vistas materializadas, Teorema CAP, ETL.

B. Análisis de Requisitos

B.1 Contexto de Ecommify

Ecommify es un marketplace de comercio electrónico inspirado en el modelo operacional de Olist, la principal plataforma de comercio electrónico brasileña. El sistema permite que vendedores independientes (sellers) registren productos en un catálogo unificado, que clientes (customers) realicen pedidos (orders) con múltiples ítems (order_items), efectúen pagos en una o varias modalidades (order_payments) y emitan reseñas tras la entrega (reviews). El dataset Brazilian E-Commerce de Olist (2018), con 99.441 pedidos, 112.650 ítems, 103.886 pagos y 100.150 reseñas, sirve como banco de pruebas representativo del comportamiento real de la plataforma.

Tras el análisis exploratorio de datos (EDA) previo (Equipo, 2026), se identificaron características que motivan una arquitectura híbrida: alta integridad referencial requerida en transacciones financieras, 41,1 % de nulos en comentarios de reseña que justifican esquema flexible, 1,0 millones de registros geoespaciales que ameritan índices 2dsphere, y patrones de consulta analítica sobre la tabla orders que se benefician de vistas materializadas.

B.2 Requisitos Funcionales

Tabla 1

Requisitos Funcionales de Ecommify

ID	Requisito Funcional	Entidades involucradas	Motor de datos
RF-01	Registrar nuevos clientes con datos personales y dirección	customers, geolocation	PostgreSQL
RF-02	Mantener catálogo de productos con atributos variables por categoría	products, categories	PostgreSQL + MongoDB
RF-03	Procesar pedidos transaccionales con múltiples ítems atómicamente	orders, order_items	PostgreSQL
RF-04	Soportar pagos con métodos múltiples y cuotas	order_payments	PostgreSQL
RF-05	Recibir reseñas opcionales del cliente tras entrega	reviews	MongoDB
RF-06	Calcular costo de envío basado en distancia cliente-depósito	customers, geolocation, warehouses	PostgreSQL (PostGIS)
RF-07	Búsqueda de productos tolerante a errores tipográficos	products	PostgreSQL (pg_trgm)
RF-08	Generar dashboards analíticos: ventas por categoría/mes, segmentos RFM	orders, order_items, payments (MV)	PostgreSQL (MV) + MongoDB
RF-09	Mantener histórico geográfico para análisis de cobertura	geolocation	MongoDB (2dsphere)
RF-10	Soportar promociones por rango temporal	products, promotions	PostgreSQL (TSTZRANGE)
RF-11	Audit trail de cambios en órdenes	orders, audit_logs	PostgreSQL (JSONB)
RF-12	Vista consolidada de orden completa para dashboards	orders_summary	MongoDB (proyección ETL)

Nota. Elaboración propia basada en el modelo operacional de Olist (2018) y el análisis del EDA (Equipo, 2026).

B.3 Requisitos No Funcionales

Tabla 2

Requisitos No Funcionales — Atributos de Calidad

ID	Atributo de calidad	Requisito	Métrica / Umbral
RNF-01	Rendimiento OLTP	Latencia p95 de inserción de orden	< 200 ms (incluye order + items + payment)
RNF-02	Rendimiento OLAP	Latencia p95 de consulta a vista materializada	< 100 ms para mv_sales_by_category_monthly
RNF-03	Disponibilidad	SLA del servicio en horario comercial	99,5 % uptime mensual
RNF-04	Consistencia transaccional	Integridad referencial financiera	100 % de FK válidas en orders, payments
RNF-05	Escalabilidad horizontal	Soporte de pedidos concurrentes	≥ 1,2 TPS sostenidos sin errores
RNF-06	Recuperación ante fallos	Recovery Time Objective (RTO)	< 30 s en failover de PostgreSQL primary
RNF-07	Durabilidad	Recovery Point Objective (RPO)	0 transacciones perdidas (synchronous_commit ON)
RNF-08	Seguridad	Hashing de contraseñas y cifrado de campos sensibles	bcrypt cost 12; pgp_sym_encrypt para tokens
RNF-09	Mantenibilidad	Versionado del esquema	Scripts DDL versionados en Git; migrations con timestamp
RNF-10	Capacidad de almacenamiento	Tamaño máximo en plan gratuito	PostgreSQL ≤ 500 MB (Supabase), MongoDB ≤ 512 MB (Atlas M0)

B.4 Identificación de Entidades del Dominio

Se identifican nueve entidades en el dominio del problema, distribuidas en tres categorías: entidades fuertes (independientes con identidad propia), entidades débiles (existen solo en relación con una entidad fuerte) y entidades de soporte (catálogos y referencias).

- **Entidades fuertes (5):** Customer, Order, Product, Seller, Review.
- **Entidades débiles (2):** OrderItem (depende de Order y Product), Payment (depende de Order).
- **Entidades de soporte (2):** Category (catálogo de traducción), Geolocation (referencia geográfica por código postal).

Nota de ajuste arquitectónico: A diferencia del modelo original, en este diseño la entidad Seller NO mantiene relación de FK con Geolocation. La geolocalización se aplica únicamente al Customer para el cálculo de costos de envío cliente-depósito. Los vendedores se gestionan sin requerimiento geográfico en esta fase inicial.

B.5 Tabla de Volúmenes Esperados

Tabla 3

Volúmenes de Datos Esperados — Dataset Olist y Proyección Ecommify

Entidad	Volumen Olist	Crecimiento anual	Tamaño estimado	Patrón de acceso predominante
customers	99.441	+30 %	12 MB	OLTP (lectura por customer_id)
orders	99.441	+30 %	24 MB	OLTP (escritura) + OLAP (rangos temporales)
order_items	112.650	+35 %	18 MB	OLTP por order_id + analítico por product/seller
order_payments	103.886	+30 %	12 MB	OLTP por order_id
reviews (MongoDB)	100.150	+25 %	15 MB	Lectura masiva (esquema flexible)
products	32.951	+15 %	8 MB	Lectura masiva + búsqueda fuzzy
sellers	3.095	+10 %	1 MB	OLTP por seller_id (sin geolocation)
geolocation (MongoDB)	1.000.163	+5 %	120 MB	Lectura geoespacial (2dsphere)
categories	71	0 %	< 100 KB	Lectura referencial
orders_summary (MongoDB)	~99.441 docs	+30 %	60 MB	Lectura analítica masiva

Nota. Los tamaños estimados se calculan asumiendo registros típicos del esquema relacional 3FN sin índices. El crecimiento anual proyectado se basa en tendencias del e-commerce brasileño (ABComm, 2024).

C. Diseño Conceptual

C.1 Modelo Entidad-Relación

El modelo conceptual representa las nueve entidades de Ecommify con sus atributos principales y las cardinalidades entre ellas, siguiendo la notación clásica de Chen (1976). El diagrama ER completo en alta calidad se encuentra en el repositorio del proyecto bajo `docs/diagrams/er_postgresql_olist.html` y su versión exportable en `docs/diagrams/er_postgresql_olist.svg`.

C.2 Descripción Detallada de Entidades

Tabla 4

Entidades del Dominio — Descripción, Atributos Clave y Tipo

Entidad	Tipo	Atributos principales	Descripción y semántica
Customer	Fuerte	customer_id (PK), customer_unique_id, zip_code_prefix (FK), city, state	Persona o entidad que realiza pedidos. Identidad doble: customer_id por pedido + customer_unique_id global para análisis de recurrencia.
Order	Fuerte (hechos)	order_id (PK), customer_id (FK), order_status, order_purchase_timestamp, order_delivered_customer_date	Pedido confirmado por un cliente. Tabla de hechos central, particionada por año sobre order_purchase_timestamp.
OrderItem	Débil	order_id+order_item_id (PK), product_id (FK), seller_id (FK), price, freight_value	Cada ítem dentro de un pedido. Identidad compuesta. Vincula orden con producto y vendedor; materializa la relación N:M.
Product	Fuerte	product_id (PK), product_category_name (FK), weight_g, dimensions, specifications (JSONB)	Artículo del catálogo. Atributos físicos en columnas tipadas; specifications flexibles en JSONB por categoría.

Seller	Fuerte	seller_id (PK), seller_zip_code_prefix, city, state	Vendedor independiente registrado. SIN FK a geolocation en este diseño (decisión de alcance).
Payment	Débil	order_id+payment_sequential (PK), payment_type, payment_installments, payment_value	Cada pago de un pedido. Permite pagos mixtos y cuotas.
Review	Débil	review_id (PK), order_id (FK), review_score, comment_title?, comment_message?	Evaluación post-entrega. Campos de texto opcionales (41,1 % nulos). MongoDB por esquema flexible.
Category	Soporte	product_category_name (PK), product_category_name_english	Tabla de traducción de categorías. Elimina dependencia transitiva category_pt→category_en de products.
Geolocation	Soporte	zip_code_prefix (PK), lat, lng, city, state	Referencia geográfica por código postal. PostgreSQL: vista materializada con coords únicas por zip. MongoDB: índice 2dsphere.

C.3 Relaciones y Cardinalidades

Tabla 5

Relaciones del Modelo Conceptual

Entidad A	Entidad B	Cardinalidad	Relación	Descripción y restricción
Customer	Order	1:N	REALIZA	Un cliente puede realizar múltiples pedidos; cada pedido pertenece a un único cliente. Participación total de Order.
Order	OrderItem	1:N	CONTIENE	Una orden contiene 1 o más ítems. Identidad débil: order_item_id solo tiene sentido con order_id.
Order	Payment	1:N	PAGA	Una orden puede tener uno o varios pagos (cuotas, métodos mixtos). 96,9 % de órdenes tienen un solo pago.
Order	Review	1:0..N	EVALÚA	Una orden puede ser evaluada 0 o más veces (96,8 % tienen exactamente 1 reseña).
Product	OrderItem	1:N	APARECE_EN	Un producto puede aparecer en muchos ítems; cada ítem referencia un único producto.
Seller	OrderItem	1:N	VENDE	Un vendedor puede vender muchos ítems; cada ítem es vendido por un único vendedor.

Category	Product	1:N	CLASIFICA	Una categoría agrupa muchos productos; cada producto pertenece a 0 o 1 categoría (FK NULLable).
Geolocation	Customer	1:N	UBICA	Un código postal puede tener muchos clientes; cada cliente referencia un único zip.
Geolocation	Seller	—	— (no aplica)	RELACIÓN ELIMINADA en este alcance. Seller no requiere geolocalización.

Nota. La relación Geolocation-Seller marcada en rojo se ELIMINÓ del alcance. Esta es una decisión arquitectónica documentada en la sección F.

C.4 Restricciones de Negocio

1. **RN-01 — Integridad de pago:** La suma de `payment_value` para una orden debe ser igual al total de (`price` + `freight_value`) de sus `order_items`. Validada en aplicación.
2. **RN-02 — Secuencialidad de items:** `order_item_id` es secuencial e inicia en 1 por cada orden. Validada por `UNIQUE(order_id, order_item_id)`.
3. **RN-03 — Estado válido:** `order_status` debe pertenecer al dominio {delivered, shipped, canceled, approved, processing, unavailable, invoiced, created}.
4. **RN-04 — Puntuación de reseña:** `review_score` entre 1 y 5. CHECK constraint en PostgreSQL; validación de esquema en MongoDB.
5. **RN-05 — Valores no negativos:** `price` ≥ 0 , `freight_value` ≥ 0 , `payment_value` > 0 , `payment_installments` ≥ 1 .
6. **RN-06 — Estado de cliente:** `customer_state` debe ser código ISO de 2 caracteres de un estado brasileño.
7. **RN-07 — Ventanas de promoción:** No pueden solaparse promociones del mismo producto. EXCLUDE constraint con GIST sobre TSTZRANGE.

8. **RN-08 — Reseña post-entrega:** `review_creation_date` debe ser posterior a `order_delivered_customer_date`. Validada en aplicación.

D. Diseño Lógico — Módulo PostgreSQL

D.1 Esquema Relacional Normalizado (3FN/FNBC)

El esquema PostgreSQL aplica normalización hasta tercera forma normal con verificación de FNBC. Las nueve tablas resultantes garantizan que: (a) todos los atributos son atómicos (1FN), (b) no existen dependencias funcionales parciales sobre PK compuestas (2FN), (c) no existen dependencias transitivas vía atributos no clave (3FN), y (d) todo determinante es superclave (FNBC). El detalle del proceso de normalización se documenta en Equipo (2026, Sección 1).

D.2 DDL Preliminar — Tablas Principales con Tipos Avanzados

Los scripts DDL completos se encuentran en el repositorio bajo postgresql/schema/. A continuación se presentan las tablas principales con la aplicación de tipos avanzados y restricciones.

D.2.1 Extensiones Requeridas

```
-- =====
-- Ecommify - Módulo PostgreSQL 16
-- Script: 01_extensions.sql
-- =====

CREATE EXTENSION IF NOT EXISTS pgcrypto;          -- gen_random_uuid() + cifrado
CREATE EXTENSION IF NOT EXISTS postgis;           -- GEOGRAPHY para envíos
CREATE EXTENSION IF NOT EXISTS pg_trgm;           -- búsqueda fuzzy productos
CREATE EXTENSION IF NOT EXISTS btree_gin;         -- índices mixtos JSONB+escalar

-- Verificar instalación
SELECT extname, extversion FROM pg_extension
WHERE extname IN ('pgcrypto', 'postgis', 'pg_trgm', 'btree_gin');
```

D.2.2 Tipos Compuestos Reutilizables

```
-- =====
-- Script: 02_types.sql - Composite types reutilizables
-- =====
```

```
CREATE TYPE address AS (
    street      TEXT,
    city        VARCHAR(60),
    state       CHAR(2),
    zip_code    INTEGER,
    country     CHAR(2)
);
```

```
CREATE TYPE dimensions AS (
    length_cm   NUMERIC(6,2),
    height_cm   NUMERIC(6,2),
    width_cm    NUMERIC(6,2),
    weight_g    INTEGER
);
```

D.2.3 Tablas Maestras

```
-- =====
-- Script: 03_tables_master.sql
-- =====
```

```
-- Tabla de referencia geográfica
```

```
CREATE TABLE geolocation (
    zip_code_prefix    INTEGER PRIMARY KEY,
    geolocation_lat    DOUBLE PRECISION NOT NULL
                        CHECK (geolocation_lat BETWEEN -33.75 AND 5.27),
    geolocation_lng    DOUBLE PRECISION NOT NULL
                        CHECK (geolocation_lng BETWEEN -73.99 AND -34.73),
    geolocation_city    VARCHAR(60) NOT NULL,
    geolocation_state   CHAR(2) NOT NULL,
    location            GEOGRAPHY(POINT, 4326)
                        GENERATED ALWAYS AS
                        (ST_MakePoint(geolocation_lng,
geolocation_lat)::geography)
                        STORED
);
CREATE INDEX idx_geo_location_gist ON geolocation USING GIST (location);
```

```
-- Categorías (tabla de traducción)
```

```
CREATE TABLE product_category_name_translation (
    product_category_name    VARCHAR(100) PRIMARY KEY,
    product_category_name_english    VARCHAR(100) NOT NULL
);
```

```

-- Customers - CON FK a geolocation
CREATE TABLE customers (
    customer_id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    customer_unique_id   UUID NOT NULL UNIQUE,
    zip_code_prefix      INTEGER NOT NULL
                        REFERENCES geolocation(zip_code_prefix),
    customer_city        VARCHAR(60),
    customer_state       CHAR(2) NOT NULL,
    shipping_address     address,                      -- composite type
    created_at           TIMESTAMPTZ DEFAULT NOW(),
    updated_at           TIMESTAMPTZ DEFAULT NOW()
);
CREATE INDEX idx_customers_state ON customers (customer_state);
CREATE INDEX idx_customers_unique ON customers (customer_unique_id);

-- Sellers - SIN FK a geolocation (decisión de alcance)
CREATE TABLE sellers (
    seller_id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    seller_zip_code_prefix INTEGER NOT NULL,           -- referencia lógica, sin FK
    seller_city          VARCHAR(60),
    seller_state         CHAR(2) NOT NULL,
    created_at           TIMESTAMPTZ DEFAULT NOW(),
    updated_at           TIMESTAMPTZ DEFAULT NOW()
);
CREATE INDEX idx_sellers_state ON sellers (seller_state);

-- Products - CON JSONB para specifications + ARRAY para photos
CREATE TABLE products (
    product_id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    product_category_name VARCHAR(100)
                        REFERENCES product_category_name_translation,
    product_name_length  INTEGER,
    product_description_length INTEGER,
    product_photos_qty   SMALLINT CHECK (product_photos_qty >= 0),
    dims                dimensions,                  -- composite
    specifications       JSONB,                      -- tipo avanzado
    photos               TEXT[],                     -- tipo avanzado
    tags                 TEXT[],                     -- tipo avanzado
    created_at           TIMESTAMPTZ DEFAULT NOW(),
    updated_at           TIMESTAMPTZ DEFAULT NOW()
);
CREATE INDEX idx_products_specs_gin ON products USING GIN (specifications
jsonb_path_ops);
CREATE INDEX idx_products_tags_gin  ON products USING GIN (tags);
CREATE INDEX idx_products_cat_trgm  ON products USING GIN (product_category_name
gin_trgm_ops);

```

D.2.4 Tablas Transaccionales con Particionamiento

```

-- =====
-- Script: 04_tables_transactional.sql
-- Tabla orders PARTICIONADA por RANGE de timestamp
-- =====

```

```

CREATE TABLE orders (
    order_id          UUID NOT NULL,
    customer_id       UUID NOT NULL

```

```

REFERENCES customers(customer_id)
ON DELETE RESTRICT,
order_status          VARCHAR(20) NOT NULL
CHECK (order_status IN (
    'delivered','shipped','canceled','approved',
    'processing','unavailable','invoiced','created')),
order_purchase_timestamp  TIMESTAMPTZ NOT NULL,
order_approved_at        TIMESTAMPTZ,
order_delivered_carrier_date TIMESTAMPTZ,
order_delivered_customer_date TIMESTAMPTZ,
order_estimated_delivery_date TIMESTAMPTZ,
audit_log                JSONB, -- historial cambios
updated_at               TIMESTAMPTZ DEFAULT NOW(),
PRIMARY KEY (order_id, order_purchase_timestamp)
) PARTITION BY RANGE (order_purchase_timestamp);

-- Particiones anuales (hot 2017-2018, cold 2016)
CREATE TABLE orders_2016 PARTITION OF orders
    FOR VALUES FROM ('2016-01-01') TO ('2017-01-01');
CREATE TABLE orders_2017 PARTITION OF orders
    FOR VALUES FROM ('2017-01-01') TO ('2018-01-01');
CREATE TABLE orders_2018 PARTITION OF orders
    FOR VALUES FROM ('2018-01-01') TO ('2019-01-01');
CREATE TABLE orders_default PARTITION OF orders DEFAULT;

CREATE INDEX idx_orders_customer ON orders (customer_id);
CREATE INDEX idx_orders_status_partial ON orders (order_status)
    WHERE order_status NOT IN ('delivered','canceled');
CREATE INDEX idx_orders_updated_at ON orders (updated_at); -- para ETL

-- order_items
CREATE TABLE order_items (
    order_id          UUID NOT NULL,
    order_purchase_timestamp TIMESTAMPTZ NOT NULL,
    order_item_id     SMALLINT NOT NULL CHECK (order_item_id > 0),
    product_id        UUID NOT NULL REFERENCES products(product_id),
    seller_id         UUID NOT NULL REFERENCES sellers(seller_id),
    shipping_limit_date TIMESTAMPTZ NOT NULL,
    price             NUMERIC(10,2) NOT NULL CHECK (price >= 0),
    freight_value     NUMERIC(10,2) NOT NULL CHECK (freight_value >= 0),
    PRIMARY KEY (order_id, order_purchase_timestamp, order_item_id),
    FOREIGN KEY (order_id, order_purchase_timestamp)
        REFERENCES orders(order_id, order_purchase_timestamp)
        ON DELETE RESTRICT
);
CREATE INDEX idx_items_product ON order_items (product_id);
CREATE INDEX idx_items_seller ON order_items (seller_id);

-- order_payments
CREATE TABLE order_payments (
    order_id          UUID NOT NULL,
    order_purchase_timestamp TIMESTAMPTZ NOT NULL,
    payment_sequential SMALLINT NOT NULL CHECK (payment_sequential >= 1),
    payment_type       VARCHAR(15) NOT NULL
        CHECK (payment_type IN (
    'credit_card','debit_card','boleto','voucher','not_defined')),
    payment_installments SMALLINT NOT NULL CHECK (payment_installments >= 1),
    payment_value       NUMERIC(10,2) NOT NULL CHECK (payment_value > 0),
    PRIMARY KEY (order_id, order_purchase_timestamp, payment_sequential),
    FOREIGN KEY (order_id, order_purchase_timestamp)

```

```

        REFERENCES orders(order_id, order_purchase_timestamp)
    );
CREATE INDEX idx_pay_type ON order_payments (payment_type);

-- promotions (nueva tabla con TSTZRANGE)
CREATE TABLE promotions (
    promotion_id    UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    product_id      UUID REFERENCES products(product_id),
    discount_pct    NUMERIC(5,2) CHECK (discount_pct BETWEEN 0 AND 100),
    period          TSTZRANGE NOT NULL,
    EXCLUDE USING GIST (product_id WITH =, period WITH &&)
);
CREATE INDEX idx_promo_period ON promotions USING GIST (period);

```

D.3 Aplicación de Tipos Avanzados Justificada

Tabla 6

Tipos Avanzados Aplicados en el Esquema PostgreSQL de Ecommify

Tabla.Columna	Tipo	Índice	Justificación
products.specifications	JSONB	GIN jsonb_path_ops	73 categorías con atributos heterogéneos. Schema flexible evita columnas NULL masivas.
products.photos	TEXT[]	GIN	Lista ordenada de URLs. Operador @> para "tiene foto X"; preserva orden.
products.tags	TEXT[]	GIN	Búsqueda multi-tag con operador && (OR lógico). Sin tabla intermedia.
promotions.period	TSTZRANGE	GIST	Ventana temporal de promoción. Operadores @>, && con índice nativo.
customers.shipping_address	address (composite)	—	Reutilización del tipo en múltiples tablas. Encapsula 5 atributos.
products.dims	dimensions (composite)	—	Agrupar peso y dimensiones físicas. Tipo reutilizable en envíos.
orders.audit_log	JSONB	—	Historial de cambios de estado. Schema dinámico (campos varían por evento).
geolocation.location	GEOGRAPHY(POINT,4326)	GIST (PostGIS)	Generada (STORED) desde lat/lng. Permite ST_Distance, ST_DWithin sin recálculos.

D.4 Extensiones Adoptadas y Justificación

- **PostGIS** — cálculo de distancia GEOGRAPHY entre cliente y depósito central (warehouse) para estimación de costo de envío. Sin geolocalización de sellers en este alcance.
- **pg_trgm** — búsqueda de productos por nombre de categoría tolerante a errores tipográficos (operador % con índice GIN gin_trgm_ops).
- **pgcrypto** — hashing bcrypt de contraseñas de aplicación y cifrado pgp_sym_encrypt de campos sensibles. Provee gen_random_uuid().
- **btree_gin** — habilitación de índices mixtos GIN sobre JSONB + columnas escalares para consultas combinadas (ej: specifications @> X AND product_category_name = Y).

D.5 Estrategia de Particionamiento

La tabla orders se particiona por RANGE sobre order_purchase_timestamp con particiones anuales. Esta estrategia se justifica por dos hallazgos del EDA: (a) el 95 % de los pedidos se concentran en 2017-2018, lo que beneficia el partition pruning en consultas analíticas de rango temporal; y (b) la tabla soporta hot/cold storage, moviendo orders_2016 a un tablespace de archivo con almacenamiento económico cuando el dataset crezca. Las tablas order_items y order_payments NO se particionan directamente; al heredar la FK compuesta (order_id, order_purchase_timestamp), las consultas JOIN aprovechan el pruning de orders.

D.6 Vistas Materializadas y Triggers

Las vistas materializadas para dashboards y los triggers de mantenimiento se documentan en el repositorio bajo `postgresql/schema/05_materialized_views.sql` y `06_triggers.sql`. Las MV principales son:

- **mv_sales_by_category_monthly** — ventas mensuales por categoría con refresh CONCURRENT semanal (domingo 02:00).
- **mv_customer_segments** — segmentación RFM (Recency, Frequency, Monetary) con refresh semanal (lunes 04:00).
- **mv_seller_performance** — ranking de vendedores por volumen y satisfacción promedio.

E. Diseño Lógico Preliminar — Módulo MongoDB

E.1 Colecciones Principales

El módulo MongoDB de Ecommify consta de cuatro colecciones que complementan al esquema relacional PostgreSQL. La asignación se fundamenta en el análisis CAP por entidad y en los hallazgos del EDA.

Tabla 7

Colecciones MongoDB de Ecommify

Colección	Origen	Patrón	Índice clave	Propósito
reviews	CSV directo	Documentos standalone	Texto (comment_msg)	Reseñas con campos opcionales (41,1 % sin comentario).
geolocation	CSV directo	GeoJSON + 2dsphere	2dsphere(location)	1 M+ coords. Búsqueda por proximidad (near, geoWithin).
products_catalog	ETL ← PG products	Embedding category	_id ← product_id	Catálogo desnormalizado con categoría embebida para lectura.
orders_summary	ETL ← PG (5 tablas)	Embedding + array	hashed(_id) + idx state	Vista analítica desnormalizada. Shard key hashed(order_id).

Decisión de alcance: Inicialmente NO se incluye una colección sellers_geo. La geolocalización aplica solo a customers (en PostgreSQL para envíos) y geolocation collection (en MongoDB para análisis).

E.2 Esquemas de Documentos — Ejemplos JSON

E.2.1 Colección reviews

```

{
  "_id": ObjectId("6650a3..."),
  "review_id": "alb2c3d4-...",           // UUID
  "order_id": "x9y8z7w6-...",           // referencia lógica a
PG
  "review_score": 5,
  "review_comment_title": "Excelente producto",           // opcional
  "review_comment_message": "Llegó antes de lo esperado.", // opcional (41.1%
nulos)
  "review_creation_date": ISODate("2018-05-12T14:30:00Z"),
  "review_answer_timestamp": ISODate("2018-05-13T09:15:00Z")
}

// índices
db.reviews.createIndex({ order_id: 1 })
db.reviews.createIndex({ review_score: 1 })
db.reviews.createIndex({ review_comment_message: "text" }) // full-text

```

E.2.2 Colección geolocation

```

{
  "_id": ObjectId("6650b4..."),
  "zip_code_prefix": 1310,
  "city": "São Paulo",
  "state": "SP",
  "location": {
    "type": "Point",
    "coordinates": [-46.6333, -23.5505]           // [lng, lat] GeoJSON
  }
}

// índices
db.geolocation.createIndex({ zip_code_prefix: 1 }, { unique: true })
db.geolocation.createIndex({ location: "2dsphere" }) // crítico

```

E.2.3 Colección products_catalog

```

{
  "_id": "plq2r3s4-...",           // = product_id de PG
  "category": {                     // embedded
    "name_pt": "casa_decoracao",
    "name_en": "home_decor"
  },
  "weight_g": 1200,
  "length_cm": 30.5, "height_cm": 15.0, "width_cm": 20.0,
  "photos_qty": 4,
  "name_length": 47, "description_length": 312,
  "etl_updated_at": ISODate("2026-05-23T03:00:00Z")
}

```

```
// índices
db.products_catalog.createIndex({ "category.name_en": 1 })
db.products_catalog.createIndex({ etl_updated_at: 1 }) // para ETL incremental
```

E.2.4 Colección *orders_summary*

```
{
  "_id": "o5t6u7v8-...", // = order_id de PG
  "status": "delivered",
  "purchase_date": ISODate("2018-07-15T11:20:00Z"),
  "customer": { // embedded subdoc
    "customer_id": "c9d0e1f2-...",
    "state": "SP",
    "city": "São Paulo"
  },
  "items": [ // array of embedded
    { "product_id": "p1...", "seller_id": "s2...",
      "price": 89.90, "freight_value": 12.50 },
    { "product_id": "p3...", "seller_id": "s2...",
      "price": 45.00, "freight_value": 0.00 }
  ],
  "payment_total": 147.40,
  "payment_type_main": "credit_card",
  "review_score": 4, // null si sin reseña
  "etl_updated_at": ISODate("2026-05-23T03:15:00Z")
}

// índices y sharding
db.orders_summary.createIndex({ purchase_date: 1 })
db.orders_summary.createIndex({ "customer.state": 1 })
db.orders_summary.createIndex({ etl_updated_at: 1 })

sh.shardCollection("ecommmify.orders_summary", { _id: "hashed" })
```

E.3 Patrones de Modelado Aplicados

Tabla 8

Patrones de Modelado MongoDB en Ecommify

Patrón	Aplicado en	Justificación	Trade-off
Embedded Document	orders_summary.customer	customer 1:1 con order; siempre se lee junto.	Update doble si customer cambia (acceptable: cambios raros).

Array of Embedded	orders_summary.items	items 1:N; cardinalidad baja (avg 1.13).	Tamaño documento < 16 MB (límite MongoDB). Aquí muy seguro.
Subset Pattern	orders_summary (no full data)	Solo campos para dashboards, no detalles operativos.	Datos completos siguen en PG; doble query si se necesita.
Computed Pattern	orders_summary.payment_total	Suma precomputada en ETL.	Reduce CPU en cada consulta; sincronía depende de refresh.
Schema Versioning	etl_updated_at en todas	Permite ETL incremental y rollback.	Campo adicional por documento (overhead mínimo).
Reference	reviews.order_id	order_id como referencia lógica (no FK).	La aplicación valida; no hay integridad nativa.
Geospatial Point	geolocation.location	GeoJSON Point con índice 2dsphere.	PostGIS sería más potente pero no disponible en Atlas M0.

E.4 Justificación: ¿Qué Datos van a MongoDB vs. PostgreSQL?

La asignación de cada entidad a un motor sigue cinco criterios objetivos derivados del EDA: (1) porcentaje de nulos en campos clave, (2) requerimiento de integridad referencial estricta, (3) consistencia financiera requerida, (4) volumen de lectura predominante, (5) flexibilidad de esquema. Los detalles cuantitativos se encuentran en la sección F.1.

F. Decisiones Arquitectónicas Justificadas

F.1 Matriz de Decisión por Entidad

Tabla 9

Matriz de Decisión Cuantitativa por Entidad — PostgreSQL vs. MongoDB

Entidad	% Nulos	FK estricta	Consist. req.	Vol. lectura	Esquema flex.	Decisión	CAP
orders	2.7%	A	A	M	B	PostgreSQL	CP
order_items	0%	A	A	M	B	PostgreSQL	CP
order_payments	0%	A	A	B	B	PostgreSQL	CP
customers	0%	A	M	M	B	PostgreSQL	CP
sellers	0%	A	M	B	B	PostgreSQL	CP
reviews	41.1%	B	B	A	A	MongoDB	AP
geolocation	0%	B	B	A	B	MongoDB	AP
products	0.8%	M	M	A	A	Mixta PG+MDB	CP/AP
categories	0%	M	M	M	B	PostgreSQL	CP
orders_summary	N/A	B	B	A	A	MongoDB	AP

Nota. A=Alto, M=Medio, B=Bajo. Azul=PostgreSQL (CP), Verde=MongoDB (AP),

Naranja=Mixta. La asignación responde al análisis CAP por entidad.

F.2 Análisis del Teorema CAP

El Teorema CAP (Brewer, 2000; Gilbert & Lynch, 2002) establece que un sistema distribuido no puede garantizar simultáneamente Consistencia (C), Disponibilidad (A) y Tolerancia a Particiones de red (P). Dado que las particiones son inevitables en sistemas

distribuidos reales, el diseño se reduce a elegir entre sistemas CP y AP. La arquitectura de Ecommify asigna cada motor según la prioridad CAP del dominio de datos:

- **PostgreSQL como sistema CP** — garantiza consistencia transaccional ACID para todos los datos financieros y de identidad. Sacrifica disponibilidad durante failover (~30 s estimados con Patroni). Justificación: una orden duplicada o un pago no registrado tendría impacto financiero directo e irreversible.
- **MongoDB como sistema AP** — prioriza disponibilidad y escalabilidad horizontal mediante sharding. Acepta consistencia eventual en lecturas de réplicas secundarias. Justificación: una reseña con 30 segundos de retraso en propagarse no constituye un error de negocio; las consultas geoespaciales toleran inconsistencia transitoria.

F.3 Trade-offs Identificados y Estrategias de Mitigación

Tabla 10

Trade-offs de la Arquitectura Híbrida y Mitigaciones

Trade-off	Impacto	Estrategia de Mitigación
Duplicación de datos (products en PG + products_catalog en MDB)	Mayor uso de almacenamiento; necesidad de mantener sincronía.	ETL upsert con etl_updated_at; trigger PG ON UPDATE products dispara sync. Aceptable: products es 32K registros (~8 MB).
Latencia de ETL para orders_summary	Datos en MDB están 1-24 h desactualizados.	Aceptable para dashboards. Para consultas en tiempo real, app va directo a PG.
Sin FK nativa MongoDB ↔ PostgreSQL	order_id huérfano en reviews si se borra de PG.	Política ON DELETE RESTRICT en PG impide borrar orders con dependencias. Soft-delete en su lugar.
Failover PG bloquea escrituras	Pérdida de disponibilidad de OLTP durante ~30 s.	Patroni con failover automático; lecturas redirigen a standby. Aceptado por SLA 99.5 %.

Lecturas secundarias MDB con replication lag	Cliente puede ver review propia con retraso de segundos.	readConcern: majority para campos críticos; readPreference: primary cuando consistencia importa.
Sharding no disponible en Atlas M0	No se puede escalar horizontalmente en plan gratuito.	Sharding emulado en Docker local; migración a Atlas M10 (\$60/mes) cuando crezca volumen.
Límite 500 MB Supabase	No cabe el dataset completo + tablas adicionales.	Cargar muestra de geolocation (100K filas representativas) en Supabase; dataset completo en local Docker.

F.4 Flujos de Sincronización (ETL)

La sincronización entre PostgreSQL y MongoDB es unidireccional (PG → MongoDB) mediante un pipeline ETL Python con tres flujos diferenciados:

Tabla 11

Flujos de Sincronización ETL entre PostgreSQL y MongoDB

Flujo	Fuente PostgreSQL	Destino MongoDB	Frecuencia	Mecanismo
F1	orders+order_items+payments+customers (JOIN)	orders_summary	Batch noche 03:00 + incremental horario	Python pandas + PyMongo upsert por _id
F2	products + categories	products_catalog	Tiempo real (trigger PG → cola)	Trigger AFTER UPDATE → notify → worker
F3	reviews CSV	reviews	Carga única inicial	PyMongo insert_many (sin ETL ongoing)
F4	geolocation CSV	geolocation	Carga única inicial	PyMongo insert_many con deduplicación por zip

G. Anexos

G.1 Diccionario de Datos Completo

El diccionario de datos completo con las 14 entidades (9 tablas PostgreSQL + 4 colecciones MongoDB + 1 tabla auxiliar warehouses) y los 96 campos detallados (nombre, tipo, longitud, descripción, restricciones) se encuentra en el repositorio bajo: docs/Diccionario_Datos_Ecommify.tsv (formato TSV importable a Excel). El archivo incluye filtros por motor (PostgreSQL / MongoDB) y por tabla.

G.2 Scripts SQL Preliminares

La estructura completa de scripts en el repositorio bajo postgresql/schema/ es:

- `01_extensions.sql` — Creación de extensiones (pgcrypto, postgis, pg_trgm, btree_gin).
- `02_types.sql` — Composite types reutilizables (address, dimensions).
- `03_tables_master.sql` — Tablas maestras (geolocation, customers, sellers, products, categories).
- `04_tables_transactional.sql` — Orders (particionada), order_items, order_payments, promotions.
- `05_materialized_views.sql` — mv_sales_by_category_monthly, mv_customer_segments, mv_seller_performance.
- `06_triggers.sql` — set_updated_at() y aplicación a todas las tablas con updated_at.

- `07_indexes.sql` — Índices adicionales (B-tree, GIN, GIST, BRIN).
- `08_jobs.sql` — Programación `pg_cron` de VACUUM, REFRESH MV y creación de particiones.

G.3 Estructura del Repositorio

```

optimizacion-y-diseno-base-de-datos/
├── README.md
├── docs/
│   ├── Documento_Tecnico_Disenio.pdf          ← este documento
│   ├── Investigacion_PostgreSQL_Avanzado.pdf
│   ├── Presentacion_Ejecutiva.pdf
│   ├── Diccionario_Datos_Ecommify.tsv
│   └── diagrams/
│       ├── er_postgresqlolist.html
│       ├── er_mongodbolist.html
│       └── arch_diagram_v3.png
├── postgresql/
│   ├── schema/                                ← 8 scripts DDL
│   ├── seed_data/
│   │   ├── load_olist_csv.py                  ← ETL Python para carga inicial
│   │   └── sample_specifications.json         ← seed JSONB samples
│   └── queries/
│       ├── analytical_queries.sql
│       ├── postgis_examples.sql
│       └── trgm_search_examples.sql
├── mongodb/
│   ├── schema/
│   │   ├── 01_collections.js                  ← createCollection con validators
│   │   ├── 02_indexes.js                     ← 2dsphere, text, hashed
│   │   └── examples/
│   │       ├── reviews.json
│   │       ├── geolocation.json
│   │       ├── products_catalog.json
│   │       └── orders_summary.json
│   └── etl/
│       ├── pg_to_mongo_orders_summary.py
│       └── pg_to_mongo_products_catalog.py
└── notebooks/
    ├── 01_Data_Exploration_Analysis.ipynb    ← EDA completo
    ├── 02_PostgreSQL_Load_Validation.ipynb
    └── 03_MongoDB_Load_Validation.ipynb
  
```

Referencias

- Brewer, E. A. (2000). Towards robust distributed systems [Keynote]. *Proceedings of the 19th Annual ACM Symposium on Principles of Distributed Computing*, 7.
<https://doi.org/10.1145/343477.343502>
- Chen, P. P.-S. (1976). The entity-relationship model—Toward a unified view of data. *ACM Transactions on Database Systems*, 1(1), 9–36. <https://doi.org/10.1145/320434.320440>
- Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6), 377–387. <https://doi.org/10.1145/362384.362685>
- Gilbert, S., & Lynch, N. (2002). Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services. *ACM SIGACT News*, 33(2), 51–59.
- MongoDB, Inc. (2025). *Data modeling introduction. MongoDB Manual*.
<https://www.mongodb.com/docs/manual/core/data-modeling-introduction/>
- MongoDB, Inc. (2025). *Sharding. MongoDB Manual*.
<https://www.mongodb.com/docs/manual/sharding/>
- Olist. (2018). *Brazilian E-Commerce public dataset by Olist* [Conjunto de datos]. Kaggle.
<https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>

PostGIS Project. (2025). *PostGIS 3.4 documentation*. <https://postgis.net/documentation/>

PostgreSQL Global Development Group. (2025). *Data types*. *PostgreSQL 16 Documentation*.
<https://www.postgresql.org/docs/16/datatype.html>

PostgreSQL Global Development Group. (2025). *Server programming: Extending SQL*.
PostgreSQL 16 Documentation. <https://www.postgresql.org/docs/16/extend.html>

PostgreSQL Global Development Group. (2025). *Table partitioning*. *PostgreSQL 16 Documentation*. <https://www.postgresql.org/docs/16/ddl-partitioning.html>

Equipo, D. (2026). *Análisis exploratorio de datos del ecosistema de e-commerce brasileño: Dataset Olist 2016-2018* [Informe académico]. Universidad de La Sabana.